



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: M.C Rene Adrian Davila Perez

Asignatura: Programación orientada a objetos

Grupo: 7

*No de
Práctica(s):* 1

Integrante(s): 323036285
323279866
323209432
323161891
323063609
314306908

*No. De Lista o
brigada* 3

Semestre: 2027-1

*Fecha de
entrega:* 28/ 08 / 2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	3
3. Desarrollo	5
4. Desarrollo	5
5. Resultados	8
6. Conclusiones	9

1. Introducción

Al iniciar en Java, a los estudiantes se les propone una meta, en éste caso, hacer un programa primitivo de “Paint”, llegando a intimidar a alguien nuevo que sabe muy poco del lenguaje.

Para resolver el problema anterior, se usan los LLM, para darle una idea al estudiante de cómo realizar su programa, surgiendo un nuevo problema, el LLM usa problemas muy complejos, intimidando aún más al estudiante, ya que implementa cosas que el estudiante no conoce.

Igualmente, al iniciar en un nuevo lenguaje de programación, a los estudiantes se les complica comprender los fundamentos de éste, complicando así la forma de implementar una calculadora sencilla que pueda resolver problemas como adición, sustracción, producto, cociente, potencia, raíz y módulo.

Resolver este último problema nos ayudará a ampliar nuestro conocimiento en Java, adquiriendo nuevas técnicas para resolver problemas simples, y mientras avanza el tiempo ir realizando el proyecto final y seguir entrenando nuestro LLM para ser más útil conforme a lo que necesitamos.

Objetivos

- Comprender el funcionamiento de una calculadora básica en lenguaje Java, para expandir los conocimientos sobre lógica y sintaxis.
- Verificar lo que recomienda un “LLM” acerca del proyecto final, para darse ideas de como realizarlo y comenzar a discernir lo que es útil.

2. Marco Teórico

Para entrenar a nuestra IA es necesario dar un contexto. En este caso le preguntamos cómo se puede generar un `paint` en el lenguaje de Java.

Un `paint` es un método utilizado para dibujar elementos gráficos en una ventana. Con él se pueden dibujar líneas, círculos, texto, imágenes, figuras geométricas, entre otros. El método `paint()` le indica al programa qué debe dibujar en una superficie gráfica. Se utiliza mucho en Java AWT/Swing para crear gráficos y aplicaciones visuales.

Entre sus aplicaciones se encuentran la creación de animaciones, videojuegos 2D sencillos, visualización de datos, etc. [2]

Para generar nuestra calculadora en Java utilizamos diferentes librerías y algunas estructuras de control para poder leer el código más fácilmente.

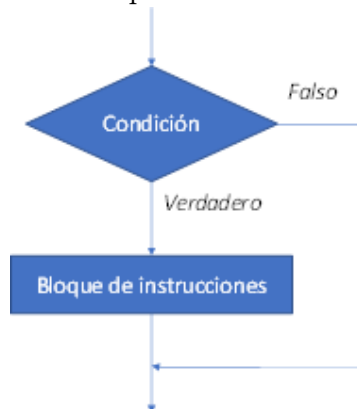
1. Clases.

Para generar nuestra calculadora en Java utilizamos diferentes librerías y algunas estructuras de control para poder leer el código más fácilmente.

- a) `java.util`: Clase de utilidad general para el programador. Este paquete contiene, por ejemplo, la clase `Scanner` utilizada para la entrada por teclado de diferentes tipos de datos, la clase `Date` para el tratamiento de fechas, etc.
- b) `java.math`: Contiene herramientas para manipulaciones matemáticas. [1]

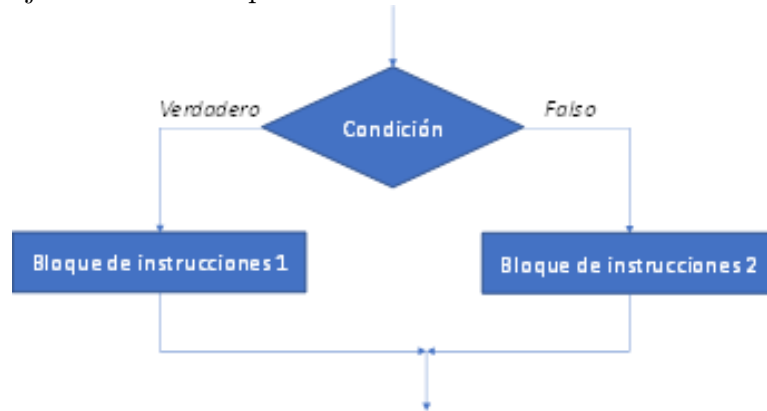
2. Estructuras de control.

- a) **Estructura if**: La función `if` es la más básica de las estructuras de control de flujo. Evalúa una condición y ejecuta un bloque de instrucciones en el caso de que la condición sea verdadera.



- b) **Estructura if-else**: La instrucción `if-else` tiene un segundo camino en el caso de que la condición sea falsa. Es decir, si es falsa, pasamos a

ejecutar otro bloque de instrucciones.



- c) **Estructura if-else if**: Esta estructura es muy parecida a las dos anteriores, con la diferencia de que empleamos varias condiciones.

```
SI (condición-1) ENTONCES
    instrucciones-1
SINO
    SI (condición-2) ENTONCES
        instrucciones-2
    SINO
        SI (condición-3) ENTONCES
            instrucciones-3
        FIN SI
    FIN SI
FIN SI
```

- d) **Estructura switch**: Esta estructura se utiliza para seleccionar un bloque de instrucciones a ejecutar, en función del valor de una expresión. Dependiendo del valor que se le dé, se va al caso que corresponda.

Funciona de la siguiente manera:

- Se evalúa la expresión de la sentencia **switch**.
- El valor de la expresión es comparado con los valores de cada caso.
- Si hay coincidencia, se ejecuta el bloque asociado.
- Las sentencias **break** (para salir del caso) y **default** (si hay un caso diferente a los demás) son opcionales.

La sintaxis es la siguiente:

```
SEGÚN expresión HACER

    CASO x:
        bloque de instrucciones
```

CASO y:
bloque de instrucciones

DE OTRO MODO:
bloque de instrucciones

FIN SEGÚN

[3]

3. Desarrollo

4. Desarrollo

1. Inicio proyecto "Paint primitivo"

Para este punto de la práctica se usó un prompt y se analizaron las diferentes respuestas:

- **Prompt utilizado**

Mi profesor nos mostró la salida de un programa estilo paint, donde se elige una figura (línea, cuadrado, círculo o rombo) y también se debe escoger si es sólida (se colorea el interior de la figura) o sólo el contorno. Además, se debe elegir el color para la figura. Existen dos opciones de borrado: sólo la figura anterior o borrar toda la pantalla.

a) **Enfoque Orientado a Objetos (Vectorial)**

El lienzo no almacena píxeles, sino una lista ordenada de objetos (`List<Figura>`). Cada objeto encapsula sus datos y su propia lógica de dibujo.

1) **Modelo de Clases**

Clase base **Figura** (Abstracta):

- Atributos: (x_1, y_1) , (x_2, y_2) , color, esSólida (booleano).
- Método abstracto: `dibujar(contextoGrafico)`.

Subclases:

- **Línea:** Traza de (x_1, y_1) a (x_2, y_2) .
- **Rectángulo:** Calcula esquina superior izquierda, ancho y alto; ejecuta trazo o relleno.
- **Círculo:** Usa la caja delimitadora del rectángulo para trazar/rellenar la elipse.
- **Rombo:** Calcula los 4 puntos medios de la caja y forma un polígono cerrado.

2) **Algoritmo del Sistema**

A. Registro de trazo (Mouse):

- Presionar clic: Guardar origen (x_1, y_1) .
- Soltar clic: Guardar fin (x_2, y_2) , instanciar la subclase elegida con el color y estado de relleno actuales, agregarla a la lista y solicitar redibujado.

B. Renderizado:

- Limpiar el fondo del lienzo.
- Recorrer la lista y llamar a `figura.dibujar(contextoGrafico)` en cada objeto.

C. Opciones de borrado:

- Borrar anterior: eliminar el último elemento (`size - 1`) y redibujar.
- Borrar todo: vaciar la lista por completo y redibujar.

b) Enfoque: Matriz de Caracteres en Consola con Códigos ANSI

Estructura: Se utiliza un arreglo bidimensional `char [][]` para almacenar los caracteres del lienzo y una matriz paralela para los colores. **Pila de Estados:** Se mantiene una matriz de respaldo `lienzoAnterior` o un `ArrayList` de matrices para deshacer pasos. **Ventaja:** No requiere librerías externas; se apega al uso de arreglos, `switch-case`, ciclos y la clase `Scanner`.

c) Enfoque: Programación Orientada a Objetos y Composición

Estructura: Jerarquía de clases con una clase base abstracta `Figura` y subclases como `Linea`, `Cuadrado`, `Circulo`, `Rombo`. **Modelo de Objetos:** Cada figura encapsula coordenadas, tamaño/radio, color y un booleano `esSolido`. **Lienzo Administrador:** Una clase `Lienzo` contiene un `ArrayList<Figura>`. **Borrado:** `figuras.remove(figuras.size() - 1)` para borrar la última figura, o `figuras.clear()` para limpiar todo.

Algoritmo General del Programa (Enfoque por Matriz y Métodos)

1) Inicialización del Entorno

Definir constantes `FILAS` y `COLUMNAS`, declarar matrices `lienzo` y `lienzoAnterior`, llenar con caracteres base, instanciar `Scanner`.

2) Bucle de Ejecución Principal (do-while)

Mostrar lienzo con ciclos `for`, desplegar menú de opciones, leer entrada con `sc.nextInt()`.

3) Selección de Figura y Dibujo

Respalda estado, capturar atributos (figura, coordenadas, dimensiones, relleno, color), rasterizar matemáticamente:

- Línea: iterar entre (x_1, y_1) y (x_2, y_2) .
- Cuadrado: recorrer rango de filas y columnas, pintar interior o contorno.

- Círculo: calcular distancia euclidiana

$$d = \sqrt{(f - filaCentro)^2 + (c - colCentro)^2}$$

pintar según condición.

- Rombo: calcular distancia Manhattan

$$d_M = |f - filaCentro| + |c - colCentro|$$

pintar según condición.

4) **Subproceso de Borrado**

Copiar `lienzoAnterior` sobre `lienzo` para deshacer, o limpiar toda la matriz para reiniciar.

5) **Terminación**

Al elegir "Salir", finalizar el ciclo `do-while` y cerrar `Scanner`.

Al comparar las respuestas, se observó que la principal diferencia entre los modelos fue el formato y la información. Pues un modelo dio el algoritmo detalladamente paso a paso, mientras que otro dio un enfoque más técnico, entregando fragmentos de código sin quitar la explicación, mientras otro modelo optó por una descripción mucho más teórica, enfocándose en explicar más los detalles técnicos del programa.

2. Calculadora

El programa se ejecuta en el método estático `main`. Mediante la importación de la clase utilitaria `Scanner` (`java.util`) para la lectura de datos desde la consola (`System.in`), se ilustra el manejo de memoria en Java al coexistir tipos primitivos de punto flotante y enteros (`double`, `int`) alojados en el `Stack` con la referencia al objeto del flujo de entrada.

El flujo principal se organiza a través de una estructura de selección múltiple `switch-case` con sentencias `break` y cláusula `default` para despachar las distintas operaciones aritméticas, apoyándose en el método estático `Math.pow` para el cálculo exponencial y radicular; asimismo, se incorporan estructuras condicionales `if-else` que actúan como validaciones de precondition para interceptar divisiones o residuos con divisor cero y raíces de base negativa, evitando indeterminaciones y resultados no numéricos (NaN) derivados de la aritmética en punto flotante.

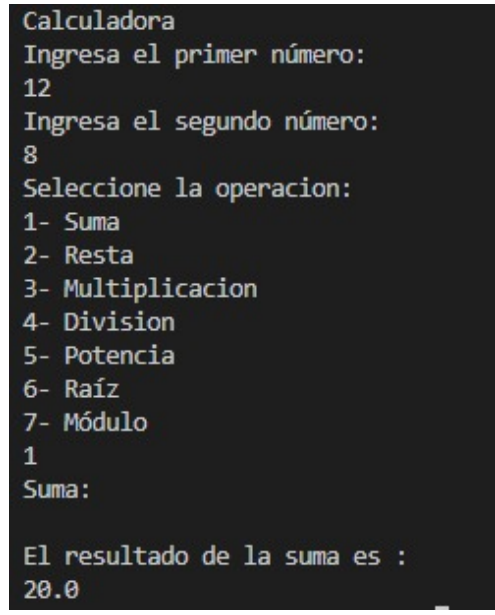
Finalmente, la ejecución concluye con el cierre explícito del flujo de entrada (`scan.close()`) para prevenir fugas de recursos en el sistema operativo.

5. Resultados

Tras concluir el desarrollo del código, realizamos las pruebas de validación necesarias para asegurar los resultados.

```
Calculadora
Ingresa el primer número:
12
Ingresa el segundo número:
8
Seleccione la operacion:
1- Suma
2- Resta
3- Multiplicacion
4- Division
5- Potencia
6- Raíz
7- Módulo
1
Suma:

El resultado de la suma es :
20.0
```

 raiz.png

Con base en los requerimientos iniciales de esta primera práctica, se le solicita al usuario que ingrese dos números base, sobre los cuales se ejecutan las operaciones correspondientes.

Si bien este flujo cubría la entrega mínima especificada, decidimos integrar un menú interactivo para optimizar la experiencia del usuario.

De este modo, la aplicación permite seleccionar de forma explícita la operación que se desea ejecutar de entre el catálogo de opciones disponibles.

A continuación se mostrarán distintos ejemplos.

```
Resta:

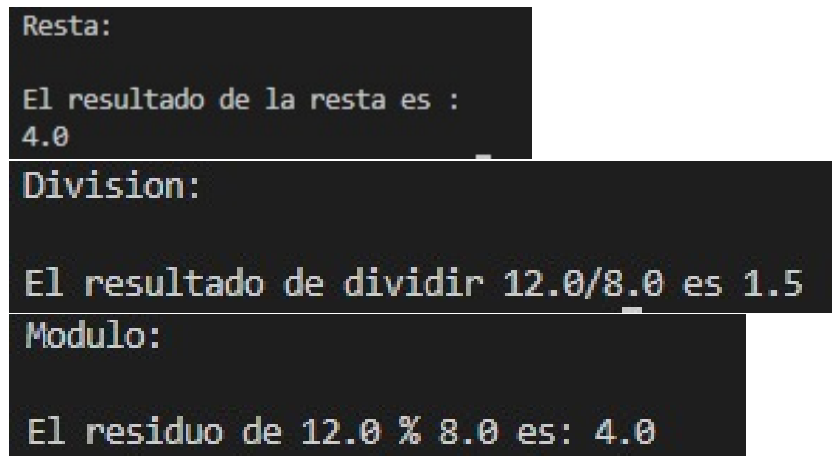
El resultado de la resta es :
4.0

Division:

El resultado de dividir 12.0/8.0 es 1.5

Modulo:

El residuo de 12.0 % 8.0 es: 4.0
```



```
Multiplicacion:
```

```
El resultado de la multiplicación es: 96.0
```

```
Potencia:
```

```
El número 12.000000 elevado a la potencia 8.000000 es: 429981696.000000
```

```
Raiz:
```

```
Resultado: 1.364261601821366
```

A partir de los dos valores de entrada, el usuario puede elegir cualquier cálculo disponible dentro del programa.

Es importante aclarar que la función de suma no se incluye en la demostración visual, ya que su desarrollo fue predefinido durante la sesión.

6. Conclusiones

El primer objetivo de la práctica se cumplió ya que se evaluó el algoritmo proporcionado por los LLM para un programa tipo Paint y se analizó en equipo, descubriendo diferencias entre las respuestas de cada LLM. El segundo objetivo se cumplió con el desarrollo de una calculadora en java, donde en la captura de datos se usó la clase Scanner de la biblioteca java.util. Asimismo, se usaron estructuras de control que ayudaron a implementar el código: se empleó switch para las siete opciones del menú, y las condicionales if e if-else para validar operaciones, previniendo algunos errores como la división entre cero.

Referencias

- [1] M. De Educación y FP. 6.4.- Librerías Java. / PROG03. Contenidos Unidad 3. https://sarreplec.caib.es/pluginfile.php/10221/mod_resource/content/3/64_librerias_java.html. Accedido: 28 de agosto de 2026. s.f.
- [2] OpenAI. Explicación del método paint() en Java y su relación con la programación orientada a objetos. <https://chatgpt.com/c/6a905b5b-34a4-83e8-a04b-7fdbfb34ac8d>. Accedido: 28 de agosto de 2026. 2026.
- [3] Throphic. Estructuras de control en Java. https://dat-science.com/estructuras-de-control-en-java/#Estructura_if. Accedido: 28 de agosto de 2026. 2025.